■ AlgoPulse: The Complete Pitch Deck Script & Jury Defense Playbook

Comprehensive Presentation Rehearsal, Deep Tech Stack Architecture, and Jury Q&A; Defense Master Guide

---

Presenter: Rohith Bheemini | **Ecosystem:** Chrome MV3 + VS Code + Next.js 14 + Gemini Flash + Supabase
Status: Production Verified (19/19 Tests Passed, 15 Routes Prerendered, Zero Build Errors)

1. Executive Presentation Script (Word-for-Word Walkthrough)

This rehearsal script covers your 8-to-10 minute pitch slide-by-slide. Speak in a confident, conversational, and natural human tone. Maintain eye contact with the jury and use open gestures.

Slide 1: Title & The Hook (0:00 - 1:00)

> *Good morning, respected judges and professors. Imagine you're preparing for a software engineering interview. You spend 45 minutes on a LeetCode problem, you click Submit, and the screen lights up bright green: 'Accepted'. You feel relieved. But in a real technical interview with Google or Microsoft, that exact code would get you rejected in five minutes. Why? Because you wrote a quadratic brute-force solution that passed only because the judge's test cases were small. The platform told you your code executed—it never told you if your approach was sound engineering. Today, we present AlgoPulse: an automated pedagogical code reviewer and skill intelligence platform that evaluates how developers think, uncovers paradigm blind spots, and guarantees long-term retention.*

Slide 2: The Origin Story — The 'Accepted' Trap (1:00 - 2:00)

> *We built AlgoPulse out of pure personal frustration, observing three fundamental flaws: First, the false sense of security: LeetCode and GFG only give binary Pass/Fail verdicts. They never reveal whether your quadratic solution should have used a single-pass hash map or two pointers. Second, the context-switching gap: we grind algorithms in web browsers, but we build real software in VS Code. Our learning history and editor work were completely disconnected. And third, the forgetting curve: you solve 100 problems this month, but two months later in an interview room, you can't recall the sliding window pattern. Human memory decays unless systematically refreshed right before extinction.*

Slide 3: Market Gap Analysis — Why This Approach vs. Others (2:00 - 3:00)

> *People ask: 'Why not just use LeetCode's judge or paste into ChatGPT?' Here is why: Traditional judges measure milliseconds on shared cloud servers, which fluctuate by 50ms due to server load noise, not algorithmic merit. Meanwhile, raw ChatGPT spoils the full solution instantly, killing the cognitive struggle essential for interview recall. AlgoPulse takes a pedagogical approach: it lives directly inside your editor, evaluates your approach on a calibrated 4-pillar rubric, and provides Socratic 3-tier hints that guide your thinking without spoiling the answer.*

Slide 4: Everyday Value in Daily Developer Workflows (3:00 - 4:00)

How does this help a developer every day? First: zero friction. You don't copy-paste code. You click 'Analyze Code' on LeetCode or hit Ctrl+Alt+A in VS Code. Second: it trains you to think like a Senior Engineer by catching hidden memory allocations before they hit production. Third: it generates an authentic, verified public portfolio transcript (/verify) proving your 16-week consistency and readiness to recruiters without leaking secret tokens.

Slide 5 & 6: System Architecture & In-Editor AST Extraction (4:00 - 5:30)

Under the hood, our client layer features a Chrome Manifest V3 extension and a VS Code desktop extension. To bypass browser sandbox isolation without fragile DOM scraping, we built custom runtime injection bridges directly into Monaco's window.monaco.editor.getModels() on LeetCode, window.ace on GeeksforGeeks, and CodeMirror on HackerRank. In VS Code, we interface natively with the active TextEditor buffer. Evaluations are processed in under 2 seconds via Google's Gemini Flash engine and synchronized to our Next.js 14 backend, which mirrors data across Supabase PostgreSQL and local JSON storage for 100% offline reliability.

Slide 7: The 4-Pillar Calibrated Rubric & 3-Tier Socratic Hints (5:30 - 6:30)

Our evaluation uses a calibrated 100-point rubric: 35 points for Algorithmic Optimality (did you pick the mathematically optimal paradigm?), 25 points for Time Complexity (distance from theoretical optimal Big-O), 20 points for Space Complexity (auxiliary allocations vs in-place mutations), and 20 points for Cleanliness and Edge Cases. Our 3-tier progressive hints coach the developer: Tier 1 nudges an invariant ('Notice the input is sorted'), Tier 2 suggests the mental model ('Consider two pointers'), and Tier 3 blueprints state transitions—preserving the student's 'Aha!' moment.

Slide 8 & 9: 14-Paradigm Matrix, Spaced Repetition & Senior Curveballs (6:30 - 8:00)

Instead of generic tags like 'Array', AlgoPulse maps submissions to 14 core technical interview archetypes—Sliding Window, Monotonic Stack, Two Heaps, Dynamic Programming, and Graph Traversals—calculating a dynamic Interview Readiness Index. It incorporates the Ebbinghaus forgetting curve decay formula to alert you when key patterns are 'Due Today' right before memory decay occurs. Finally, our Senior Interview Follow-Up Curveballs prepare candidates for real interview pivots: 'What if N = 10^12 and exceeds RAM?' or 'Can you solve this with strict O(1) space?' with full reference architectures.

Slide 10, 11 & 12: Live Demo, Free-Tier Stack & Conclusion (8:00 - 9:30)

AlgoPulse operates on a 100% free-tier architecture: Gemini Flash free tier, Next.js locally or on Vercel Hobby, and Supabase free tier. It transforms competitive coding from mindless test grinding into verified software craftsmanship. Let me now demonstrate this live across LeetCode, VS Code, and our real-time dashboard.

2. The 3-Minute Live Demonstration Blueprint

Follow this exact step-by-step sequence during the live presentation. It guarantees maximum visual impact in minimum time:

Step 1: Web Dashboard Landing Page (30 Seconds)

Open http://localhost:3000. Highlight the Google 4-color ambient glowing lighting. Scroll down to the Interactive Review Demo widget. Click 'Brute Force O(N^2)' to show the red 42/100 score, then click 'Optimal Hash Map O(N)' to show it jump to 100/100. Say: 'This proves AlgoPulse evaluates algorithmic invariants, not just test cases.'

Step 2: Chrome Extension on LeetCode (45 Seconds)

Switch to Chrome on LeetCode's Two Sum problem. Point to the floating AlgoPulse widget. Click 'Analyze Code'. Show the slide-over drawer sliding in smoothly. Walk through the 4-pillar breakdown. Click 'Reveal Tier 1 Observation' to demonstrate progressive hints without code spoiling.

Step 3: VS Code Desktop Extension (30 Seconds)

Switch to VS Code with demo.py open. Press Ctrl+Alt+A (or right-click -> 'AlgoPulse: Analyze Current Code File'). The review appears in the Activity Bar within 2 seconds. Demonstrate the 'Insert in Editor' button inserting optimal reference code.

Step 4: Real-Time Heatmap & Public Verification Sync (15 Seconds)

Switch back to http://localhost:3000/dashboard. Point to the green 'Live Polling Active' badge and show the new submission appearing in real time. Click 'Verify' to show the public authenticated transcript (/verify) and copy the shareable link.

3. Exhaustive Jury Q&A;: Deep Tech Stack Architecture

Jury members and technical professors will probe your implementation choices. Here are expert, battle-tested answers:

Q1: Why did you choose Chrome Manifest V3 instead of Manifest V2, and how do you handle MV3 service worker lifecycles?

Chrome has deprecated Manifest V2. Manifest V3 improves security by enforcing short-lived event-driven background service workers and banning remotely hosted code. Because MV3 service workers terminate after 30 seconds of inactivity, our extension avoids relying on in-memory state in the background script. Instead, we use chrome.storage.local for persisting tokens and settings, and perform direct fetch requests directly from the content script context and background workers.

Q2: How do you extract code from Monaco and Ace editors without violating Chrome's isolated world security model?

Chrome extensions run content scripts in an ISOLATED world that cannot directly access window.monaco or window.ace on the host page. Rather than using fragile DOM scraping (which breaks with obfuscated class names), we inject a lightweight bridge script into the MAIN execution world. This bridge accesses window.monaco.editor.getModels()[0].getValue(), and transmits the clean buffer string back to our content script via window.postMessage with an origin-verified handshake.

Q3: How is the VS Code extension architected, and why did you use WebviewViewProvider over simple notifications?

Our VS Code extension is built with TypeScript and the VS Code Ext API. We registered a custom WebviewViewProvider in the Activity Bar rather than transient editor notifications. This allows us to render a persistent, rich interactive review panel that maintains dark GitHub theme parity, supports collapsible hint buttons, and provides an 'Insert in Editor' command via vscode.TextEditorEdit. Document capture uses vscode.window.activeTextEditor to pull selections or full document text.

Q4: Why Next.js 14 App Router instead of a standard Express or Fastify backend?

Next.js 14 gives us full-stack cohesion: React 18 Server Components for zero-bundle-size static page generation, combined with edge-ready Route Handlers (/api/*) in a unified TypeScript project. Server-side rendering guarantees instant initial load times for the dashboard and heatmap, while built-in Route Handlers process submissions, analytics, and authentication with zero external server overhead.

Q5: How does your backend handle micro-level validation, out-of-bounds scores, and malformed JSON?

In our latest micro-level audit, we created a centralized api-utils.ts layer. First, req.json() parsing is wrapped in an isolated try/catch returning a clean 400 Bad Request if syntax is malformed. Second, all 5 score dimensions are run through a clampScore helper restricting them strictly to integers between 0 and 100. Third, text fields have strict length caps (500KB for user code, 250 chars for titles) preventing memory denial-of-service. Finally, all endpoints enforce strict CORS headers and Cache-Control: no-store.

Q6: How does the hybrid database architecture work between Supabase PostgreSQL and local storage?

We implemented a resilient dual-storage engine in storage.ts. When configured, submissions write to Supabase PostgreSQL using service role keys. Concurrently, every submission is mirrored to a local JSON file

(dashboard/data/submissions.json) using synchronous atomic file writes. If Supabase is offline or network fails, the system automatically falls back to local JSON storage with zero downtime. This guaranteed our 19/19 endpoint verification tests passed seamlessly.

Q7: How do you prompt Google Gemini to guarantee deterministic 4-pillar scores without hardcoding deprecated models?

We implemented dynamic model resolution querying Google AI Studio's ModelService.ListModels API on the fly with the user's API key. This automatically discovers active models supporting generateContent (such as gemini-3.7-flash, gemini-3.6-flash, and gemini-3.8-flash) with zero hardcoding. We invoke Gemini with a temperature of 0.2 and a strict JSON system instruction. The prompt defines explicit rubrics for each pillar (e.g. 'Optimality: 35 pts if optimal invariant chosen, 15 pts if sub-optimal, 0 if brute force'). Gemini outputs raw JSON matching our TypeScript SubmissionRecord interface with zero markdown noise, achieving sub-2-second response latency.

Q8: Why 5-second polling on the dashboard instead of WebSockets or Server-Sent Events (SSE)?

WebSockets require stateful persistent server connections, which break on serverless edge environments (like Vercel) and consume background CPU. Server-Sent Events introduce reconnection overhead on flaky client networks. Given that users submit code every few minutes, a lightweight 5-second HTTP poll fetching updated counts with Cache-Control: no-store is stateless, free-tier compatible, and produces negligible bandwidth overhead while delivering near-instant dashboard sync.

Q9: How does authentication work between the extensions and dashboard without leaking passwords?

We use secure, high-entropy Extension Bearer Tokens (e.g., ap_sec_...). Users generate this token in their private Profile page. Extensions transmit this token via Authorization: Bearer . Our resolveUserFromRequest utility verifies the token against profiles in Supabase. For public viewing, our /verify endpoint renders a zero-token public transcript showing only aggregated metrics and activity without exposing the user's secret credentials.

Q10: What is your API test coverage and how did you verify all endpoints?

We wrote an automated verification suite in test_endpoints.js that runs 19 exhaustive tests: health checks, CORS preflight (OPTIONS 204), HEAD uptime monitoring, analytics aggregation, auth fallback, query filtering (limit, platform), malformed JSON rejection (400), score clamping (185 -> 100), single resource lookup (/api/submissions/[id]), non-existent record handling (404), and RESTful deletion (DELETE 200). All 19 tests pass with 0 failures.

4. Exhaustive Jury Q&A;: Algorithmic Approach & Pedagogy

Questions on algorithmic evaluation methodology, spaced repetition formulas, and learning psychology:

Q11: Why evaluate code by algorithmic invariants rather than execution time in milliseconds?

Execution time on competitive programming sites is fundamentally flawed. Server container contention, background GC pauses, and CPU clock throttling cause identical solutions to swing from 30ms to 90ms. More critically, small test inputs allow O(N^2) algorithms to pass within the 1-second timeout. AlgoPulse evaluates the invariant structure: recognizing whether the problem requires a monotonic stack, sliding window, or hash table, scoring mathematical optimality regardless of hardware clock cycles.

Q12: Why not just use GitHub Copilot or ChatGPT?

Copilot and ChatGPT are generative completion engines—they write the solution for the developer. In education, this induces the 'illusion of competence': students feel they understand the problem because the AI solved it. In interview settings, candidates freeze because they never developed the mental recall. AlgoPulse is a pedagogical reviewer: it refuses to spoil code, providing progressive hints that preserve the critical struggle needed to form lasting neural pathways.

Q13: How did you calculate the weights for the 4-Pillar Scoring Rubric (35, 25, 20, 20)?

The rubric mirrors Tier-1 tech interview evaluation scorecards: Optimality (35%) is weighted highest because picking the wrong paradigm (e.g. recursion instead of DP) is an automatic interview rejection. Time Complexity (25%) evaluates asymptotic scaling. Space Complexity (20%) penalizes wasteful heap allocations when in-place pointer manipulation is possible. Cleanliness & Edge Cases (20%) evaluates boundary conditions (empty inputs, integer overflow) and idiomatic coding style.

Q14: Why did you establish a 14-Paradigm Competency Matrix instead of using LeetCode's tags?

LeetCode's tags are broad data structures (e.g. 'Array', 'String'). An Array problem could be solved via Two Pointers, Sliding Window, Monotonic Stack, Binary Search, or Prefix Sum. Tagging it as 'Array' provides zero guidance on how to solve it. Our 14 archetypes represent the actual algorithmic patterns evaluated by FAANG interviewers, allowing us to accurately compute a candidate's Interview Readiness Index and highlight exact blind spots.

Q15: How does your Ebbinghaus Spaced Repetition decay formula work in practice?

Hermann Ebbinghaus established that human retention decays exponentially unless active recall occurs. Our algorithm computes: Decay % = min(100, Days Elapsed * 16 - Score Bonus). Solutions scoring >=90 receive a score bonus that slows decay. When decay exceeds 45% or 4 days pass, the problem enters 'Due Today'. This schedules reviews at the precise moment of impending memory extinction, maximizing retention with minimum review time.

Q16: Why are Senior Interviewer Follow-Up Curveballs important?

In real L5/L6 senior interviews, writing working code is merely checkpoint 1. Senior interviewers immediately challenge your assumptions: 'What if the dataset is 10^12 elements and exceeds RAM?' or 'What if this must run concurrently across multiple worker threads?' AlgoPulse presents Scale, Constraint, and Concurrency curveballs with expandable reference architectures, elevating candidates from junior coders to senior software engineers.

Q17: How does your platform uphold academic integrity in university classrooms?

Many universities ban ChatGPT because students use it to cheat on assignments. AlgoPulse solves this by enforcing progressive Socratic scaffolding. Tier 1 only reveals structural invariants without naming algorithms. Tier 2 suggests patterns. Tier 3 gives conceptual blueprints without code. The reference code is locked behind deliberate user

confirmation, ensuring students engage in genuine problem-solving.

Q18: How does AlgoPulse handle difficulty across different platforms (e.g., GeeksforGeeks uses 'School' and 'Basic', while HackerRank and LeetCode use Easy/Medium/Hard)?

Platforms have inconsistent taxonomy: GeeksforGeeks has School, Basic, Easy, Medium, and Hard, while HackerRank and LeetCode use Easy, Medium, and Hard. AlgoPulse implements canonical normalization in both its browser extractors and backend API: 'School' and 'Basic' are automatically normalized into 'Easy'. Our backend API accepts raw 'Basic' or 'School' from any platform and stores them as 'Easy', ensuring the 3-tier difficulty distribution and heatmap metrics remain mathematically coherent across all platforms.

Q19: How does the AI model categorize difficulty in VS Code where there are no HTML badges?

In local editor files, there is no web DOM badge to scrape. Our Gemini prompt explicitly requests an 'assessed_difficulty' field ('Easy' | 'Medium' | 'Hard'). The AI evaluates the algorithmic techniques and data structures required by the code—e.g. single-pass frequency maps are assessed as 'Easy', sliding windows and tree traversals as 'Medium', and multi-state dynamic programming or monotonic deques as 'Hard'. This enables local VS Code work to seamlessly contribute to the candidate's multi-platform analytics.

Q20: How does AlgoPulse guarantee user data isolation and persistent analytics across sessions?

Every submission is strictly keyed to the authenticated user ID (user_id). When a new user creates an account, all dashboards and paradigm meters initialize strictly at zero with zero artificial mock data. Submissions pushed from LeetCode, GeeksforGeeks, or VS Code write synchronously to both the user's Supabase account and local encrypted storage, persisting consistently across sign-out and sign-in cycles.

Q21: How does the web extension handle Chrome extension reloads without crashing the host page?

When an unpacked extension is reloaded in Chrome Developer mode, Chrome invalidates the background communication channel for open tabs. AlgoPulse defensively intercepts 'Extension context invalidated' errors in overlay.tsx, preventing uncaught runtime exceptions, and renders an interactive 'Refresh Page (F5) to Reconnect' button directly in the UI drawer for one-click recovery.

5. Live Jury Challenge Scenarios ('Test This Right Now!')

If the jury challenges you to test specific edge cases live on screen, use these pre-rehearsed steps:

Challenge A: 'Show me Brute Force O(N^2) vs Optimal O(N) scoring right now.'

1. Navigate to http://localhost:3000.
2. Scroll down to the Interactive Review Demo card.
3. Click 'Brute Force O(N^2)': point out the 42/100 score, the red time complexity alert, and the why_suboptimal explanation.
4. Click 'Optimal Hash Map O(N)': point out the score jumping to 100/100 with green complexity bars.
5. Verbal Punchline: 'This demonstrates that AlgoPulse evaluates algorithmic invariants rather than binary pass/fail.'

Challenge B: 'Write and test a Python algorithm in VS Code right in front of us.'

1. Switch to VS Code. Create or open test.py.
2. Paste a Two Sum hash map solution:
class Solution:
def twoSum(self, nums, target):
seen = {}
for i, n in enumerate(nums):
if target - n in seen: return [seen[target - n], i]
seen[n] = i
return []
3. Press Ctrl+Alt+A.
4. Show the panel in the Activity Bar displaying 95/100 score within 2 seconds.
5. Switch to browser at http://localhost:3000/dashboard: show the submission synced live to the heatmap.

Challenge C: 'Prove that your progressive hints don't spoil the code.'

1. In the review panel, navigate to Progressive Socratic Hints.
2. Click 'Reveal Observation (Tier 1)': show that it says 'Notice that the input array is already sorted' without naming any algorithm.
3. Click 'Reveal Pattern (Tier 2)': show that it suggests 'Two Pointers converging inward'.
4. Verbal Punchline: 'The student gets the cognitive nudge to solve the problem themselves without having the code handed to them.'

Challenge D: 'Show me your Public Verified Portfolio and prove it doesn't leak secrets.'

1. Navigate to http://localhost:3000/verify.
2. Point to the verified candidate transcript: 16-week heatmap, streaks, 14-paradigm matrix, and readiness score.
3. Click 'Copy Public Link' (green check appears).
4. Open a fresh Incognito browser window and paste the link.
5. Verbal Punchline: 'Recruiters and mentors can verify a candidate's consistency and readiness score without seeing private API tokens or login credentials.'

Challenge E: 'Demonstrate live analysis on GeeksforGeeks Practice (e.g. Kadane's Algorithm).'

1. Open GeeksforGeeks Practice: https://www.geeksforgeeks.org/problems/kadanes-algorithm-1587115620/1.
2. Paste an O(N) Kadane's solution in Python3.
3. Click 'AlgoPulse Review' on the bottom right -> Click 'Analyze Code'.
4. Highlight auto-detected platform ('GeeksforGeeks'), difficulty ('Medium'), and 95+ score recognizing 'Dynamic Programming / Greedy'.
5. Open http://localhost:3000/dashboard: show the submission immediately counted under GFG and the DP paradigm meter incremented.

6. Emergency Troubleshooting & Rapid Recovery Runbook

Symptom	Root Cause	10-Second Fix
Port 3000 already in use	Previous node process running	Run: Stop-Process -Name node -Force; cd dashboard; npm run start
WiFi drops during demo	Venue internet disconnection	Dashboard automatically falls back to local JSON mirror (data/submissions.json); zero crash.
LeetCode/GFG editor code not reading	User not on a problem page	Ensure URL is a problem page; click 'Analyze Code' on floating pill.
Extension context invalidated	Extension reloaded in chrome://extensions	Click 'Refresh Page (F5) to Reconnect' directly in the error card.
VS Code hotkey does not fire	Active focus is in terminal	Click inside editor on code, press Ctrl+Alt+A or reload window (Developer: Reload Window).